

Polynomial Regression Assignment Using Insurance Dataset

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```
# Load Dataset
df = pd.read_csv(r"C:\5th Semester\ML\insurance.csv")
df.head()
```

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86

```
df.info()
```

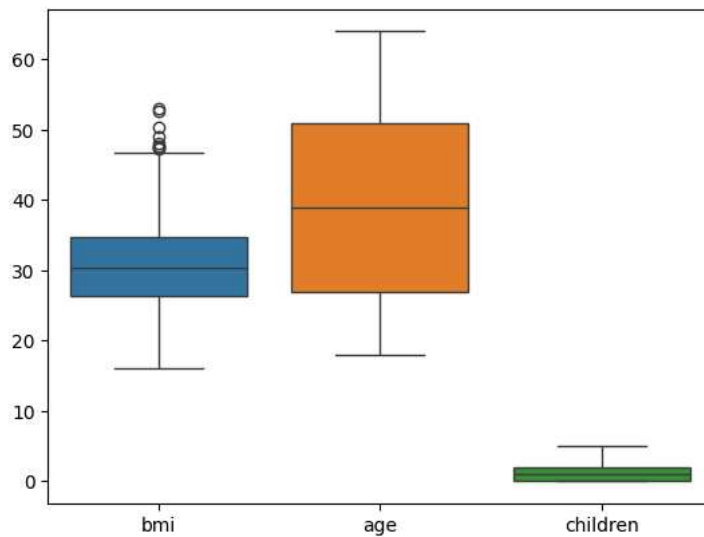
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1338 entries, 0 to 1337
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   age         1338 non-null   int64
1   sex         1338 non-null   object
2   bmi         1338 non-null   float64
3   children    1338 non-null   int64
4   smoker      1338 non-null   object
5   region      1338 non-null   object
6   expenses    1338 non-null   float64
dtypes: float64(2), int64(2), object(3)
memory usage: 73.3+ KB
```

```
df.isna().sum()
```

```
age      0
sex      0
bmi      0
children 0
smoker   0
region   0
expenses 0
dtype: int64
```

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.boxplot(data=df[['bmi', 'age', 'children']])
plt.show()
```



df

	age	sex	bmi	children	smoker	region	expenses
0	19	female	27.9	0	yes	southwest	16884.92
1	18	male	33.8	1	no	southeast	1725.55
2	28	male	33.0	3	no	southeast	4449.46
3	33	male	22.7	0	no	northwest	21984.47
4	32	male	28.9	0	no	northwest	3866.86
...	...	...	...	...	...	...	...
1333	50	male	31.0	3	no	northwest	10600.55
1334	18	female	31.9	0	no	northeast	2205.98
1335	18	female	36.9	0	no	southeast	1629.83
1336	21	female	25.8	0	no	southwest	2007.95
1337	61	female	29.1	0	yes	northwest	29141.36

1338 rows × 7 columns

```
# Features and Target
X = df.drop('expenses', axis=1)
y = df['expenses']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(1070, 6) (268, 6) (1070,) (268,)
```

```
from sklearn.preprocessing import LabelEncoder, RobustScaler, PolynomialFeatures

le_sex = LabelEncoder()
le_smoker = LabelEncoder()

X_train['sex'] = le_sex.fit_transform(X_train['sex'])
X_test['sex'] = le_sex.transform(X_test['sex'])

X_train['smoker'] = le_smoker.fit_transform(X_train['smoker'])
X_test['smoker'] = le_smoker.transform(X_test['smoker'])

X_train_region = pd.get_dummies(X_train['region'], drop_first=True)
X_test_region = pd.get_dummies(X_test['region'], drop_first=True)

X_test_region = X_test_region.reindex(columns=X_train_region.columns, fill_value=0)
```

```
X_train = pd.concat([X_train.drop('region', axis=1), X_train_region], axis=1)
X_test = pd.concat([X_test.drop('region', axis=1), X_test_region], axis=1)
```

```
# from sklearn.preprocessing import StandardScaler

# scaler = StandardScaler()
# X_train_scaled = scaler.fit_transform(X_train)
# X_test_scaled = scaler.transform(X_test)

from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Polynomial Features
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X_train_scaled)
X_poly_test = poly.fit_transform(X_test_scaled)

print(X_poly.shape)
```

```
(1070, 44)
```

```
# Train Model
model = LinearRegression()
model.fit(X_poly, y_train)
```

▼ LinearRegression ⓘ ?

► Parameters

```
# Predictions
y_pred = model.predict(X_poly_test)

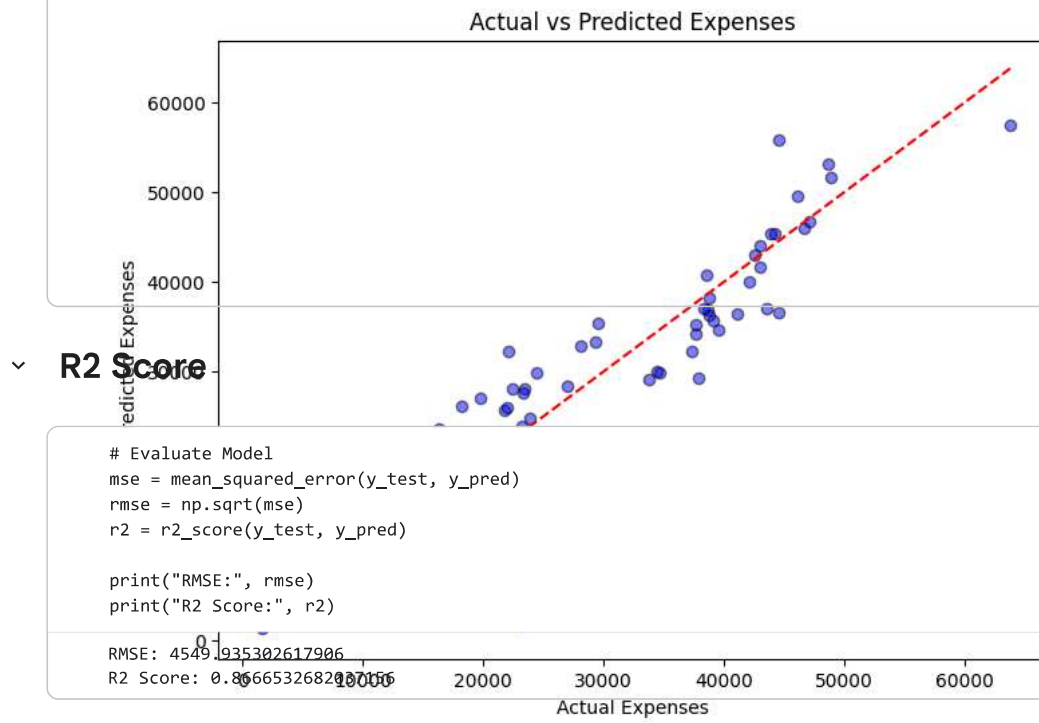
print(y_pred[:10])
```

```
[11222.39037173  6059.05557258 33312.69185932 10827.95465905
 29056.04274728  4255.02936848  2808.92934375 16031.5576474
  4106.12304491 12147.53064148]
```

▼ best fitted line

```
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.5, color='blue', edgecolor='k')

plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--')
plt.xlabel('Actual Expenses')
plt.ylabel('Predicted Expenses')
plt.title('Actual vs Predicted Expenses')
plt.show()
```



```
# Evaluate Model
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("RMSE:", rmse)
print("R2 Score:", r2)
```

RMSE: 4549.935302617906
R2 Score: 0.8666532682037156

RMSE: 4549.935302617906
R2 Score: 0.8666532682037156

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.